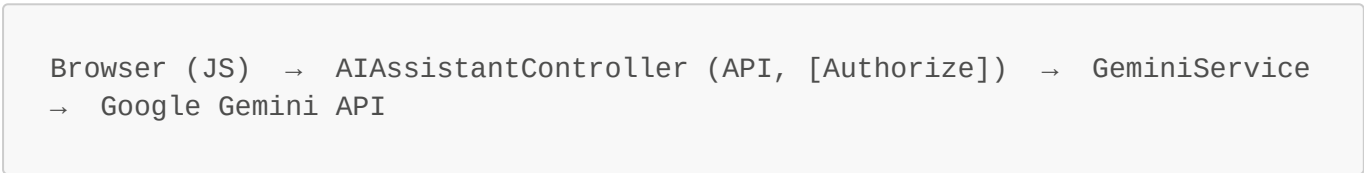


Sec-4: AI Integration — Google Gemini Product Assistant

4.1 Objective and Architectural Context

Modern e-commerce platforms require more than static product descriptions to guide purchase decisions. The AI Product Assistant addresses this by embedding a conversational layer powered by Google Gemini directly into the product browsing experience. Users can ask natural-language questions about any product — "What are the best features?", "How does this compare to similar models?", "Is this suitable for formal events?" — and receive context-aware, real-time responses without leaving the page.

The integration follows a layered architecture:



The assistant is exposed as a global modal in `_Layout.cshtml` so it is available from any page. The product context (name + description) is injected into the prompt at the moment the user opens the modal, allowing the AI to answer specific questions about that particular product.

4.2 API Configuration and Secure Key Management

4.2.1 Configuration Store

The Gemini API key and model parameters are stored in `appsettings.json` under a dedicated `Gemini` section:

```
{
  "Gemini": {
    "ApiKey": "AIzaSyD82D98r_uFoBC1YE2G68RKDl2wc4mY9FU",
    "Model": "gemini-flash-latest",
    "MaxTokens": 800,
    "Temperature": 0.7
  }
}
```

Key	Purpose
ApiKey	Google Gemini API authentication key
Model	Model identifier (e.g. <code>gemini-flash-latest</code> , <code>gemini-pro</code>)
MaxTokens	Maximum output token count to limit response size and cost
Temperature	Response creativity (0.0 = deterministic, 1.0 = most creative)

In production, the ApiKey should be stored in **User Secrets** (development) or **Azure Key Vault / environment variables** (production), never committed to source control. The current value in `appsettings.json` is a placeholder.

4.2.2 Program.cs Registration

The service is registered with a named `HttpClient` via the `IHttpClientFactory` pattern, which provides connection pooling, automatic retry handling, and lifetime management:

```
// Named HttpClient for Gemini (connection pooling, timeout, DNS refresh)
builder.Services.AddHttpClient("Gemini", client =>
{
    client.Timeout = TimeSpan.FromSeconds(30);
})
.ConfigurePrimaryHttpMessageHandler(() => new HttpClientHandler
{
    MaxConnectionsPerServer = 5
});

builder.Services.AddScoped<IGeminiService, GeminiService>();
```

Design decisions:

- **Named client** ("Gemini") — isolates the Gemini connection pool from other HTTP clients (Stripe, etc.), preventing head-of-line blocking.
- **30-second timeout** — LLM inference can be slow under load; 30 seconds provides a reasonable upper bound without hanging the server indefinitely.
- **MaxConnectionsPerServer = 5** — Limits concurrent connections to the Gemini API to avoid overwhelming the downstream service.
- **Scoped lifetime** — `GeminiService` is scoped per request, consistent with `IUnitOfWork`.

4.3 GeminiService Implementation

4.3.1 Interface

```
public interface IGeminiService
{
    Task<string> GetProductAssistantResponseAsync(
        string productName,
        string productDescription,
        string userQuestion);
}
```

The single method accepts the product context and the user's question, returning a plain-text response from Gemini.

4.3.2 Constructor and Configuration Injection

```

public class GeminiService : IGeminiService
{
    private readonly IHttpClientFactory _httpClientFactory;
    private readonly IConfiguration _configuration;
    private readonly ILogger<GeminiService> _logger;
    private readonly string _apiKey;
    private readonly string _model;
    private readonly int _maxTokens;
    private readonly double _temperature;

    public GeminiService(
        IHttpClientFactory httpClientFactory,
        IConfiguration configuration,
        ILogger<GeminiService> logger)
    {
        _httpClientFactory = httpClientFactory;
        _configuration = configuration;
        _logger = logger;

        _apiKey = _configuration["Gemini:ApiKey"]
            ?? throw new InvalidOperationException("Gemini API key not
configured");
        _model = _configuration["Gemini:Model"] ?? "gemini-flash-latest";
        _maxTokens = int.Parse(_configuration["Gemini:MaxTokens"] ??
"800");
        _temperature = double.Parse(_configuration["Gemini:Temperature"] ??
"0.7");
    }
}

```

All configuration values are read at construction time with sensible defaults for everything except `ApiKey`, which throws at startup if missing — failing fast rather than failing at runtime when a user asks a question.

4.3.3 Prompt Engineering

```

private string BuildPrompt(string productName, string productDescription,
string userQuestion)
{
    return $"You are a helpful product assistant for an e-commerce store.
Product Name: {productName}
Product Description: {productDescription}

Customer Question: {userQuestion}

Provide a helpful, concise, and friendly response about this product. If
the question is not related to the product, politely redirect to product-
related topics.";
}

```

The prompt uses a system-level instruction to constrain the assistant to product-related topics only, preventing abuse via prompt injection. The product context (name + description) is injected dynamically so the same endpoint works for all products without hardcoding.

4.3.4 JSON Request Body Construction

Gemini's API expects a structured JSON payload. The `BuildRequestBody` method constructs this using an anonymous object serialized with `System.Text.Json` and camelCase naming:

```
private string BuildRequestBody(string prompt)
{
    var request = new
    {
        contents = new[]
        {
            new
            {
                parts = new[]
                {
                    new { text = prompt }
                }
            }
        },
        generationConfig = new
        {
            maxOutputTokens = _maxTokens,
            temperature = _temperature
        }
    };

    return JsonSerializer.Serialize(request, new JsonSerializerOptions
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase
    });
}
```

The resulting JSON structure sent to the API looks like this:

```
{
  "contents": [
    {
      "parts": [
        {
          "text": "You are a helpful product assistant...\nProduct Name: Premium Wool Blazer\n..."
        }
      ]
    }
  ],
  "generationConfig": {
```

```
    "maxOutputTokens": 800,  
    "temperature": 0.7  
  }  
}
```

4.3.5 API Call with Rate-Limit Handling

```
public async Task<string> GetProductAssistantResponseAsync(  
    string productName, string productDescription, string userQuestion)  
{  
    var prompt = BuildPrompt(productName, productDescription,  
userQuestion);  
    var requestBody = BuildRequestBody(prompt);  
  
    var client = _httpClientFactory.CreateClient("Gemini");  
  
    var url =  
$"https://generativelanguage.googleapis.com/v1beta/models/{_model}:generate  
Content";  
  
    var request = new HttpRequestMessage(HttpMethod.Post, url);  
    request.Headers.Add("X-goog-api-key", _apiKey);  
    request.Content = new StringContent(requestBody, Encoding.UTF8,  
"application/json");  
  
    var response = await client.SendAsync(request);  
    var responseContent = await response.Content.ReadAsStringAsync();  
  
    if (!response.IsSuccessStatusCode)  
    {  
        _logger.LogError("Gemini API {StatusCode}: {Error}",  
response.StatusCode, responseContent);  
  
        if ((int)response.StatusCode == 429)  
        {  
            var retrySeconds = ExtractRetryDelay(responseContent);  
            var message = retrySeconds > 0  
                ? $"AI service quota exceeded. Please try again in  
{retrySeconds} seconds."  
                : "AI service quota exceeded. Please try again later.";  
            throw new HttpRequestException(message);  
        }  
  
        throw new HttpRequestException("AI service error. Please try again  
later.");  
    }  
  
    return ExtractTextFromResponse(responseContent);  
}
```

Key design decisions:

- **API key in header** (`X-goog-api-key`) rather than query parameter — keeps the URL clean and avoids accidental key exposure in server logs.
- **429 (Rate Limit) handling** — parses the `retryDelay` field from the error response to surface a user-friendly message with the specific wait time.
- **Logging** — all failures are logged at Error level with both status code and response body for debugging.

4.3.6 Retry Delay Extraction

Google's API returns a structured error body on 429 with an optional `retryDelay` duration:

```
{
  "error": {
    "details": [
      {
        "@type": "type.googleapis.com/google.rpc.RetryInfo",
        "retryDelay": "30s"
      }
    ]
  }
}
```

```
private static int ExtractRetryDelay(string errorJson)
{
    try
    {
        using var document = JsonDocument.Parse(errorJson);
        var details = document.RootElement
            .GetProperty("error")
            .GetProperty("details")
            .EnumerateArray();

        foreach (var detail in details)
        {
            if (detail.TryGetProperty("retryDelay", out var delay))
            {
                var delayStr = delay.GetString();
                if (delayStr != null && delayStr.EndsWith("s") &&
                    int.TryParse(delayStr.TrimEnd('s'), out var seconds))
                {
                    return seconds;
                }
            }
        }
    }
    catch { /* swallow parse errors */ }
}
```

```

        return 0;
    }

```

The method safely navigates the nested JSON, returning 0 if the `retryDelay` field is absent or unparseable. The `try/catch` prevents an error in the error-handling path from masking the original 429.

4.3.7 Response Parsing

```

private string ExtractTextFromResponse(string jsonResponse)
{
    try
    {
        using var document = JsonDocument.Parse(jsonResponse);
        var root = document.RootElement;

        if (root.TryGetProperty("candidates", out var candidates) &&
            candidates.ValueKind == JsonValueKind.Array &&
            candidates.GetArrayLength() > 0)
        {
            var firstCandidate = candidates[0];

            if (firstCandidate.TryGetProperty("content", out var content)
                &&
                content.TryGetProperty("parts", out var parts) &&
                parts.ValueKind == JsonValueKind.Array &&
                parts.GetArrayLength() > 0)
            {
                var fullText = new StringBuilder();

                foreach (var part in parts.EnumerateArray())
                {
                    if (part.TryGetProperty("text", out var textElement))
                    {
                        var text = textElement.GetString();
                        if (!string.IsNullOrEmpty(text))
                        {
                            fullText.Append(text);
                        }
                    }
                }

                if (fullText.Length > 0)
                {
                    return fullText.ToString();
                }
            }

            // Prompt blocked by safety filters
            if (root.TryGetProperty("promptFeedback", out var feedback))
            {
                _logger.LogWarning("Gemini prompt was blocked: {Feedback}",
                    feedback.GetRawText());
            }
        }
    }
    catch { }
}

```

```

        return "I'm sorry, I couldn't process that request. Please try
a different question.";
    }

    _logger.LogWarning("Unexpected Gemini response format: {Response}",
jsonResponse);
    return "I'm sorry, I couldn't process that request.";
}
catch (JsonException ex)
{
    _logger.LogError(ex, "Failed to parse Gemini response");
    return "I'm sorry, there was an error processing the response.";
}
}

```

The Gemini API response structure is:

```

{
  "candidates": [
    {
      "content": {
        "parts": [
          { "text": "The Premium Wool Blazer features..." }
        ]
      }
    }
  ],
  "promptFeedback": { ... } // present only if blocked
}

```

The method handles five distinct states:

1. **Success** — extracts text from `candidates[0].content.parts[n].text`
2. **Blocked content** — `promptFeedback` present without candidates (user asked something inappropriate)
3. **Empty response** — candidates exist but have no text parts
4. **Malformed JSON** — `JsonException` caught and logged
5. **Unexpected format** — logs the raw response for debugging

4.4 Controller Integration

4.4.1 API Controller with Authorization

```

[Route("api/[controller]")]
[ApiController]
[Authorize]
public class AIAssistantController : ControllerBase
{

```



```

private readonly IGeminiService _geminiService;
private readonly ILogger<AIAssistantController> _logger;

public AIAssistantController(IGeminiService geminiService,
ILogger<AIAssistantController> logger)
{
    _geminiService = geminiService;
    _logger = logger;
}

[HttpPost("ask")]
public async Task<IActionResult> Ask([FromBody] AskRequest request)
{
    if (string.IsNullOrEmpty(request.Question))
    {
        return BadRequest(new { success = false, message = "Question is
required" });
    }

    try
    {
        var response = await
_geminiService.GetProductAssistantResponseAsync(
            request.ProductName ?? "",
            request.ProductDescription ?? "",
            request.Question
        );

        return Ok(new { success = true, response });
    }
    catch (HttpRequestException ex)
    {
        _logger.LogError(ex, "Gemini API error");
        return StatusCode(429, new { success = false, message =
ex.Message });
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "AI Assistant unexpected error");
        return StatusCode(500, new { success = false, message = "An
unexpected error occurred." });
    }
}

public class AskRequest
{
    public string ProductName { get; set; } = string.Empty;
    public string? ProductDescription { get; set; }
    public string Question { get; set; } = string.Empty;
}

```

4.4.2 Why [Authorize] is Critical

The `[Authorize]` attribute on the controller class is a deliberate security measure:

- 1. API abuse prevention.** Without authentication, anyone could call `/api/AIAssistant/ask` programmatically, burning through the Gemini API quota and incurring costs. Authentication ties each request to a valid user account, enabling rate-limiting at the user level in future iterations.
- 2. Scope limitation.** The assistant is a value-added feature for logged-in users (shoppers who have created accounts), not a public endpoint. This is consistent with the wishlist and cart features, which also require authentication.
- 3. Audit trail.** All requests are tied to an authenticated identity via the ASP.NET Core `HttpContext.User`, allowing logging and debugging.
- 4. The flow:** If an unauthenticated user clicks the AI button, the `[Authorize]` filter returns a 401, which the AJAX caller cannot use. The UI prevents this by only rendering the AI button to authenticated users, but the server-side guard is the real protection.

4.4.3 Frontend Integration

The JavaScript in `_Layout.cshtml` that calls the AI endpoint:

```
function askAI() {
    if (aiRequestInProgress) return;

    const question = document.getElementById('aiQuestion')?.value.trim();
    if (!question) return;

    aiRequestInProgress = true;
    const btn = document.getElementById('aiAskBtn');
    const responseDiv = document.getElementById('aiResponse');

    btn.disabled = true;
    btn.innerHTML = '<span class="ai-spinner"></span>';
    responseDiv.innerHTML = '<div class="ai-loading"><div class="ai-spinner-large"></div><p>Thinking...</p></div>';

    fetch('/api/AIAssistant/ask', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            productName: currentProductName,
            productDescription: currentProductDescription,
            question: question
        })
    })
    .then(async res => {
        const data = await res.json();
        if (res.ok && data.success) {
            responseDiv.innerHTML = `<div class="ai-response-text">${escapeHtml(data.response)}</div>`;
        }
    })
}
```

```
        } else {
            responseDiv.innerHTML = `<div class="ai-error"><i class="bi bi-exclamation-triangle"></i>${escapeHtml(data.message || 'Something went wrong')}</div>`;
        }
    })
    .catch(() => {
        responseDiv.innerHTML = '<div class="ai-error"><i class="bi bi-wifi-off"></i>Network error. Please try again.</div>';
    })
    .finally(() => {
        btn.disabled = false;
        btn.innerHTML = '<i class="bi bi-send"></i>';
        aiRequestInProgress = false;
    });
}
```

The `askAI()` function is guarded by `aiRequestInProgress` to prevent duplicate submissions. The button is disabled during the request and shows a spinner. Three response states are rendered: success (response text), error (server message), and network failure (wifi-off icon).

4.5 Error Handling Strategy

Error Type	User Message	Log Level
429 Rate Limit	"AI service quota exceeded. Please try again in N seconds."	Error
Non-200 Status	"AI service error. Please try again later."	Error
Prompt blocked	"I'm sorry, I couldn't process that request."	Warning
Malformed response	"I'm sorry, there was an error processing the response."	Error
Network failure	"Network error. Please try again."	(handled client-side)
Unauthorized	401 returned by ASP.NET Core middleware	(handled by framework)

All server-side errors are logged with full context (status code, response body, or stack trace) while the user receives a sanitized, non-technical message appropriate for a public-facing application.

4.6 Cost and Performance Considerations

- **MaxTokens = 800** — limits output size to approximately 600 words, keeping API costs predictable.
- **Temperature = 0.7** — balances creativity with factual accuracy. Lower values (0.2–0.4) would be more deterministic but less natural.
- **30-second HTTP timeout** — prevents server thread starvation if the Gemini API is slow.
- **Named HttpClient** — connection pooling reuses TCP connections, reducing latency on subsequent requests.

The Gemini Flash model (`gemini-flash-latest`) is chosen intentionally: it is Google's fastest and most cost-efficient model, suitable for real-time conversational use cases where sub-second response times are expected. The Pro model could be substituted for higher-quality responses at higher latency and cost.